

RAZVOJ VIŠESLOJNIH APLIKACIJA



**PRIMENJENO
SOFTVERSKO
INŽENJERSTVO**

3. NEDELJA
PROJEKTNİ OBRASCI
(DESIGN PATTERN)

PROFESOR: DARKO ČAPKO
ASISTENTI: FILIP ĐORĐEVIĆ
BOGDAN DRLJEVIĆ

PROJEKTNİ OBRASCI COMMAND ADAPTER STATE

TEMA 3

PROJEKTNI OBRASCI

U PROGRAMIRANJU, PROJEKTNI OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

REŠENJA ZA OPŠTE PROBLEME KOJI SE ČESTO SREĆU U RAZLIČITIM KONTEKSTIMA RAZVOJA SOFTVERA.

Projektni obrazac nije gotovo rešenje koje je moguće ubaciti direktno u programski kod, već predstavljaju princip kako rešiti određeni problem u različitim situacijama.

Nisu povezani sa konkretnim programskim jezikom, već su osnova za implementaciju u različitim programskim jezicima.

Većina ovih obrazaca se opisuje vrlo formalno kako bi bila data osnova za reprodukciju obrazaca u više konteksta. U opisu obrazaca, prisutne su sledeće informacije:

- ❖ **Namera** - kratak opis problema i rešenja.
- ❖ **Motivacija** - dalji, detaljniji opis problema i rešenja koje omogućava obrazac.
- ❖ **Klasna struktura** - pokazuje delove/celine obrasca i odnose među njima.
- ❖ **Primer koda** - u nekom od programskih jezika, kako bi se razumela implementacija.

PROJEKTNI OBRASCI

PODELA PROJEKTNIH OBRAZACA:

● **Obrasci stvaranja (Creational Patterns)** - opisuju mehanizme stvaranja objekata. Sistem ostaje nezavistan u odnosu na stvaranje i kompoziciju objekata.

● **Strukturni obrasci (Structural Patterns)** - opisuju kompoziciju klasa i objekata kako bi se formirale kompleksnije strukture.

● **Obrasci ponašanja (Behavioral Patterns)** – opisuju interakciju i komunikaciju među objektima.

PROJEKTNII OBRAZAC COMMAND

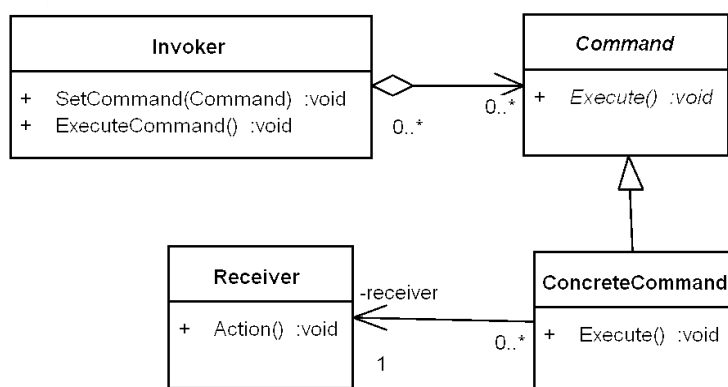
U PROGRAMIRANJU, PROJEKTNII OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA PROJEKTNIM OBRASCIMA PONAŠANJA

Namera: Enkapsulirati zahtev za izvršavanje operacije kao objekat, čime se omogućava parametrizacija klijenata različitim zahtevima, redosledno zahteva, kao i podrška za operacije koje se mogu poništiti (undo/redo)

Umesto da se direktno izvrši određena operacija, kreira se objekat-komanda, koji se potom prosleđuje na izvršenje.

Klasni dijagram:

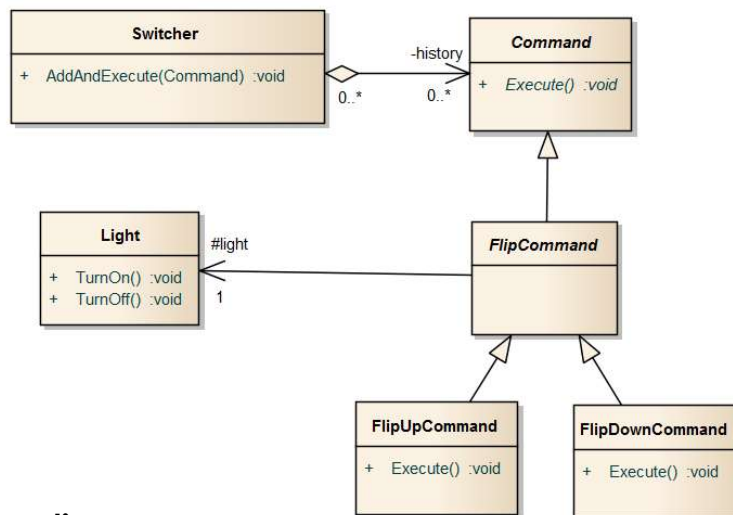


PROJEKTNII OBRAZAC COMMAND

U PROGRAMIRANJU, PROJEKTNII OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA PROJEKTNIM OBRASCIMA PONAŠANJA

Primer:



Prednosti:

- ❖ Razdvaja objekat koji poziva radnju od objekta koji je izvršava
- ❖ Omogućava izvršavanje niza komandi odjednom

Mane:

- ❖ Povećavanje broja klasa sa svakom novom komandom

PRIMER

PRIMENITI PROJEKTNI OBRAZAC COMMAND PRI REALIZACIJI SLEDEĆIH FUNKCIONALNOSTI:

- Omogućiti da se na zadati broj (0) primene sve računske operacije: sabiranje, oduzimanje, množenje i deljenje sa drugim brojem
- Omogućiti undo i redo akcije.

PROJEKTNI OBRAZAC ADAPTER

U PROGRAMIRANJU, PROJEKTNI OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA STRUKTURNIM PROJEKTNIM OBRASCIMA

Namera: Konvertovati interfejs jedne klase u drugi interfejs koji klijenti očekuju. Adapter omogućava klasama da sarađuju iako to inače ne bi mogle zbog nekompatibilnih interfejsa.

Pristupi:

Klasni adapter - oslanja se na višestruko nasleđivanje, gde je moguće redefinisati i menjanje ponašanja sa obe strane

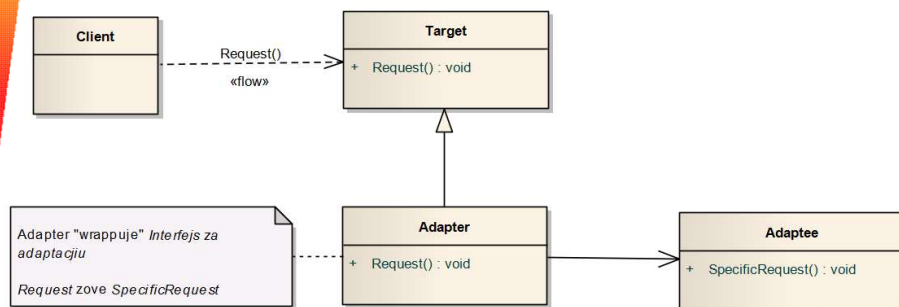
Objektni adapter - kada nije podržano višestruko nasleđivanje, jedna adapterska klasa podržava rad sa svim podklasama.

PROJEKTI OBRAZAC ADAPTER

U PROGRAMIRANJU, PROJEKTI OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA STRUKTURNIM PROJEKTNIM OBRASCIMA

Klasni dijagram:



Potrebno je koristiti više postojećih podklasa, ali je nepraktično prilagoditi njihov interfejs nasleđivanjem svake pojedinačno. Objektni adapter može prilagoditi interfejs svoje nadklase.

PRIMER

PRIMENITI PROJEKTI OBRAZAC ADAPTER PRI REALIZACIJI SLEDEĆIH FUNKCIONALNOSTI:

Osoba X je u sistemu definisala **zaposlene (Profesor, Portir, Čistačica)** na fakultetu. Svaki zaposleni od atributa ima *ime* i *prezime* i *metodu* za *ispis*.

Osoba Y je uvela klasu **Student**, nasledivši klasu **Person**, koja ima polja *firstName* i *lastName*.

Za klasu kojom je modelovan student, kreirati objektni adapter, tako da se omogući ispis imena i prezimena i za studente.

PROJEKTI OBRAZAC STATE

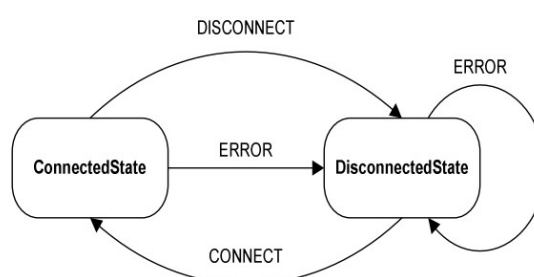
U PROGRAMIRANJU, PROJEKTI OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA PROJEKTNIM OBRASCIMA PONAŠANJA

Namera: Omogućiti objektu da menja svoje ponašanje kada se njegovo interno stanje promeni. Objekat će izgledati kao da menja svoju klasu.

Mašina stanja (State-Machine):

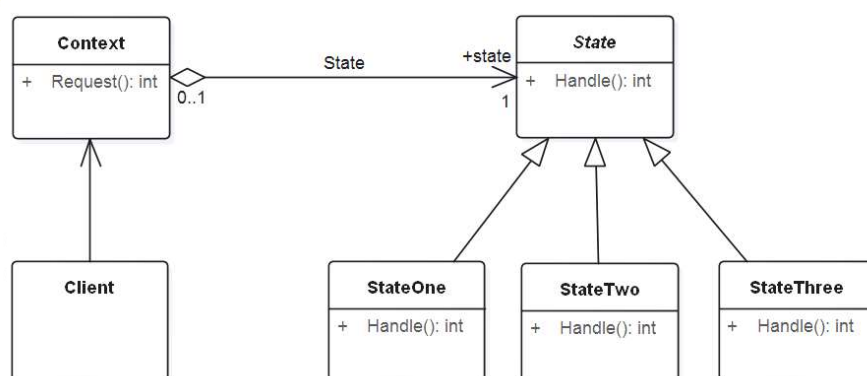
Mašina stanja, ili konačni automat, predstavlja apstraktnu mašinu, koja u jednom trenutku može biti u tačno jednom stanju, iz konačnog skupa stanja. Mašina stanja je definisana listom mogućih stanja, početnim stanjem, i uslovima za prelaz iz stanja u stanje. Prelazi iz jednog u drugo stanje se dešavaju kao odgovor na spoljašnje ulaze.



PROJEKTI OBRAZAC STATE

U PROGRAMIRANJU, PROJEKTI OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA PROJEKTNIM OBRASCIMA PONAŠANJA



Obrazac *State* implementira svako stanje kao klasu, koja implementira određeni interfejs ili nasleđuje apstraktnu klasu. Prelazi se iniciraju unutar klasa koje predstavljaju određeno stanje, pozivom odgovarajućih metoda.

PROJEKTNII OBRAZAC STATE

U PROGRAMIRANJU, PROJEKTNII OBRASCI PREDSTAVLJAJU UOPŠTENI PRISTUP REŠAVANJU ČESTIH PROBLEMA U ARHITEKTURI SOFTVERA.

PRIPADA PROJEKTNIM OBRASCIMA PONAŠANJA

Primena:

- ❖ Kada ponašanje objekta mora da se prilagodi u toku rada (*run-time*).
- ❖ U slučaju kada operacije imaju velike, višestruke uslove (*if-else*, *switch...*) koji zavise od stanja u kom se nalazi objekat. U slučaju obrasca *State*, svako grananje uslova se stavlja u posebnu klasu. Ovim se omogućuje da se svako stanje tretira kao poseban objekat, koji može varirati nezavisno od drugih objekata.

Enumeracije?

Prostiji pristup implementaciji mašine stanja je sa jednom ili više enumeracija. Obično se ne navodi kao programski obrazac, jer ne zadovoljava *open/closed* princip.

Ne predstavlja pravi objektno orijentisani pristup!

PRIMER

PRIMENITI PROJEKTNII OBRAZAC STATE PRI REALIZACIJI SLEDEĆIH FUNKCIONALNOSTI:

- Rešenje opisuje rad sa bankovnim računom - **Account**. On se može naći u tri stanja: *Standard*, *Bonus* i *Minus*.
- Na račun se mogu uplatiti i mogu se povući sredstva (*Deposit & Withdraw*). U okviru stanja se vodi računa o količini novca sa kojom se raspolaže, a definisane su i donja i gornja granica novca.
- Ukoliko je količina novca između 0 i 1000, račun se nalazi u stanju *Standard*. Ako količina novca padne ispod 0, račun prelazi u *Minus* stanje. Ako količina novca pređe 1000, račun prelazi u *Bonus* stanje.

ZADATAK

IZABRATI ADEKVATNI PROJEKTNI OBRAZAC I PRIMENITI GA PRI REALIZACIJI SLEDEĆIH FUNKCIONALNOSTI:

- **Konj** i **Pas** su modelovani sa klasama koje implemetiraju interfejs aktivnosti životinja - *IZivotinjskaAktivnost*. Aktivnosti su *OglasiSe* i *PomeriSe*.

- U softveru već postoji apstraktna klasa **Ptica** sa operacijama *Leti* i *Pevaj*. Klasu **Ptica** nasleđuju **Labud** i **Vrabac**.

- Omogućiti da sve životinje mogu da izvrše aktivnosti iz interfejsa *IZivotinjskaAktivnost*. Instancirati po jedan objekat svake životinje i pozvati metode iz interfejsa.

implementacija metoda treba da ispiše poruke na konzolnom prozoru: „Konj trči“, „Vrabac cvrkuće“ itd.